Обучение драфта EAGLE-3 с нуля: полный отчёт

Слои статьи (2/16/29) против наших (16/22/29 + `fc_norm`) для LLaMA-3.1-8B-Instruct: обучение, валидация, реальная τ и ускорение, что драфт берёт из слоёв, баг SGLang, прогноз

Команда EAGLE-3, большое обучение

15 октября 2026

Коротко

1. **Наши слои лучше и при обучении с нуля.** Реальная τ в SGLang у ours выше, чем у baseline: в среднем по пяти бенчмаркам +0,037, 95 % интервал [+0,014; +0,060], $p \approx 0{,}001$. Выигрыш сосредоточен в диалогах: MT-bench +0,08, Alpaca +0,09. Код авторов подтверждает: +0,06 / +0,05 / −0,03 на MT-bench / HumanEval / GSM8K.
2. **Преимущество устойчивое.** На валидации ours впереди во всех 35 точках из 35, с первых 16 тыс. диалогов; разрыв слегка растёт. На реальной генерации по промежуточным чекпоинтам ours впереди на MT-bench и Alpaca в 7 случаях из 7. Baseline нужно на 8–15 % больше данных для той же τ. Разрыв есть на каждой позиции цепочки, кроме первой.
3. **Почему так — видно по весам.** Baseline почти не пользуется своим нижним слоем 2: тот даёт 1,8 % выхода fc, а слой 29 — 83 %. У ours все три слоя работают: 30 / 26 / 44 %. Признаки слоя 2 в 25 раз меньше по норме, чем у слоя 29; `fc_norm` выравнивает масштабы.
4. **`fc_norm` бесплатен по скорости.** Ускорение 2,91× у ours и 2,90× у baseline; скорость повторяет τ (корреляция 0,97). Официальный драфт быстрее на диалогах (3,06× в среднем), наши — на коде и математике. Это разница данных обучения.
5. **Нашли и исправили баг SGLang 0.5.9.** Драфт со слоями не по умолчанию получал в CUDA-графах слои 2/16/29, и τ ours падала на 13 %. Исправление — в пакете `sglang_eagle31`.
6. **Прогноз.** К одной эпохе τ-оценка ours ≈4,14 против ≈4,09 у baseline, разрыв растёт. До конца эпохи около 23,5 ч H200 на вариант.

Содержание

1 Задача и постановка 3

2 Как шло обучение 4

2.1 Эпизод с NeMo . . . 6

3 Валидация: ours впереди с самого начала 7

3.1 Экономия данных . . . 9

4 Реальная τ и ускорение 9

4.1 Как устроен выигрыш 11

5 Реальная τ по ходу обучения **12**

6 Почему наши слои помогают: что драфт берёт из каждого слоя **14**

7 Баг SGLang 0.5.9: слои цели в CUDA-графах **15**

8 Прогноз: сколько ещё учить **16**

9 Выводы и что дальше **16**

A Хронология финальной стадии **17**

B Данные и воспроизводимость **17**

1 Задача и постановка

Вопрос. В исследовании выбора слоёв мы нашли, что EAGLE-3 берёт из целевой модели не лучшие слои: нижний из трёх почти не используется, а полезная зона — около 70 % глубины сети и самый верх. Но те опыты дообучали драфт, обученный под слои статьи, так что у слоёв статьи была фора. Здесь проверяем честно: оба драфта учатся с нуля, одинаково, и отличаются только входом.

Таблица 1: Условия сравнения.

	baseline	**ours**
Слои цели, подаваемые в fc	2, 16, 29 (как в статье)	16, 22, 29 ($N/2$, 0,7N, $N-3$)
Нормировка входа fc	нет	`fc_norm`: RMSNorm на каждый слой с обучаемым масштабом (как в EAGLE-3.1)
Общее		
Целевая модель	LLaMA-3.1-8B-Instruct, 32 слоя	
Данные	PerfectBlend-Regenerated-Llama-3.1-8B-Instruct: ответы сгенерированы самой LLaMA; 1 366 504 диалога в обучении, 2000 отложены, 400 из них — валидация	
Код и гиперпараметры	код авторов (SafeAILab/EAGLE, `traineagle3`), DeepSpeed fp16, AdamW (0,9; 0,95), lr 5e-5 с разогревом и линейным спадом на одну эпоху, 8 диалогов на шаг, TTT 7 шагов, клиппинг 0,5	
Точка сравнения	шаг 68 325 = 546 600 диалогов = 0,4 эпохи (решение тимлида: режем эпохи, не данные)	
Железо	одна H200, сессии SLURM до 8 ч; оба варианта на одной GPU	

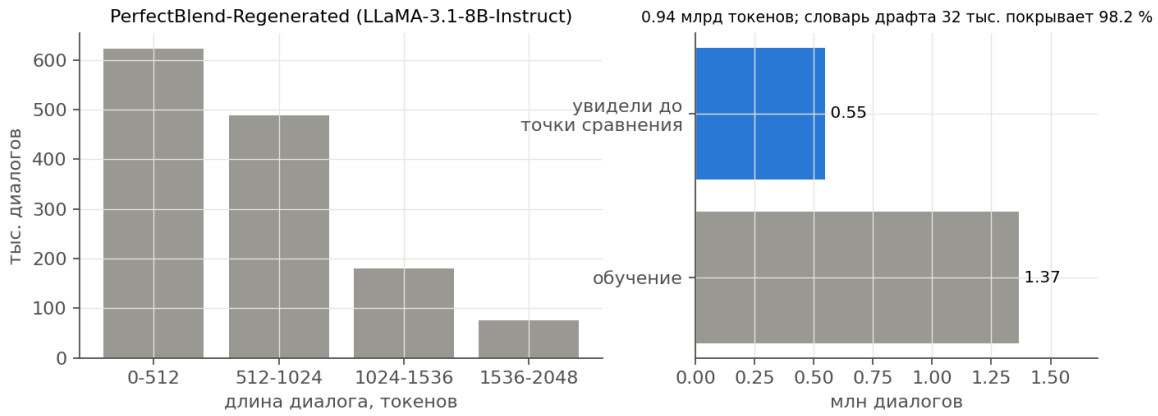


Рис. 1: Данные. Слева — длины диалогов (длиннее 2048 токенов выброшены, как у авторов). Справа — сколько данных увидели драфты до точки сравнения.

Как мерили.

- *τ-оценка на валидации* — каждые 16 тыс. диалогов, на одних и тех же 400 отложенных диалогах. Это ожидаемая длина одной цепочки драфта при измеренной точности по позициям: $\hat{\tau} = 1 +$

$\sum_k \prod_{j \le k} \text{acc}_j$. Она ниже реальной τ на дереве, но дёшева и одинакова для обоих вариантов.

- *Реальная τ* — число токенов за раунд проверки при настоящей генерации: SGLang 0.5.9, дерево статьи (8 шагов, top-k 10, 60 токенов), T = 0, batch 1, по 80 вопросов на MT-bench, HumanEval, GSM8K, Alpaca и CNN/DM. Для контроля её же мерили кодом авторов на первых трёх.
- *Ускорение* — скорость с драфтом, делённая на обычную генерацию на той же свободной GPU.
- *Интервалы* — парный bootstrap по вопросам, 10 000 повторов.

2 Как шло обучение

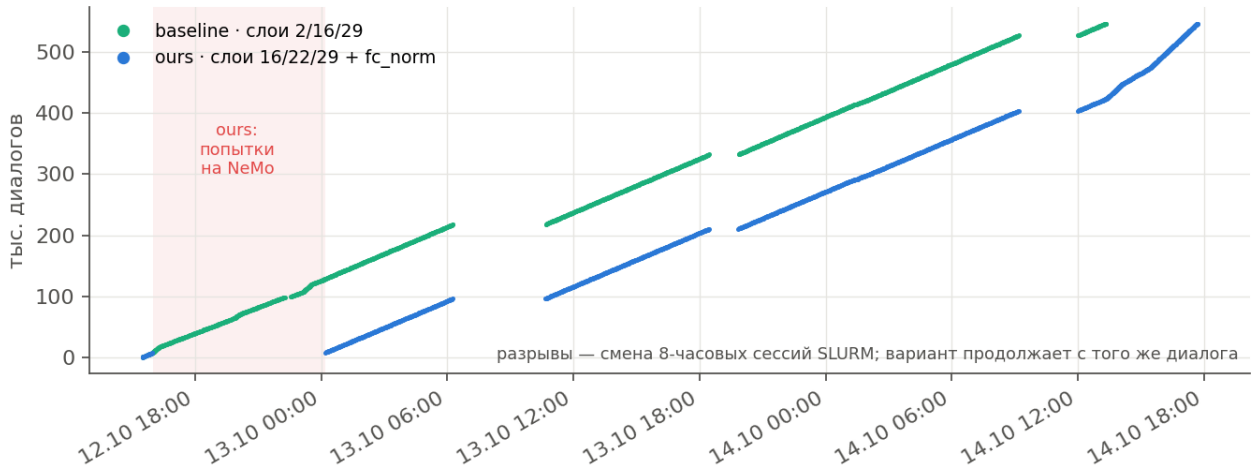


Рис. 2: Ход обучения по часам. Каждая сессия SLURM длится до 8 часов; перед концом сессии вариант сохраняется и в следующей продолжает с того же диалога. Красная полоса — время, когда ours учился рецептом NeMo (раздел 2.1).

- Оба варианта стартовали 12.10 в 15:26 на одной H200. Baseline прошёл 6 сессий и закончил 14.10 в 13:22.
- Ours сначала тоже шёл кодом авторов, но с 16:00 12.10 его перевели на рецепт NeMo AutoModel. Тот оказался нестабилен, и с 00:12 13.10 ours продолжил кодом авторов с шага 998. Закончил 14.10 в 17:42.
- Чистое время обучения — 36,9 ч у baseline и 33,3 ч у ours. Пока варианты шли вместе, каждый получал ≈4,4 диалога/с; один на GPU — ≈9,7 (рис. 3).

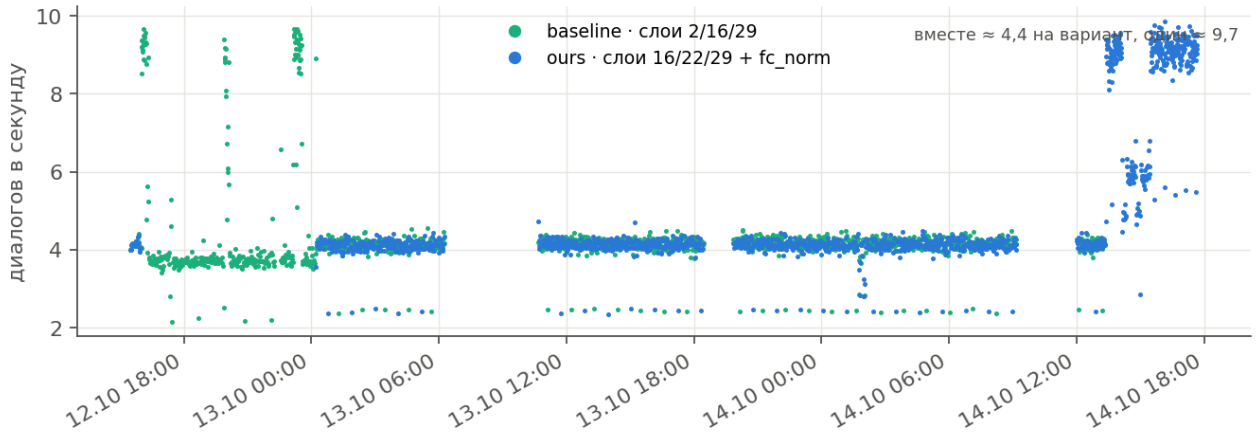


Рис. 3: Скорость обучения. После того как baseline закончил, ours ускорился вдвое.

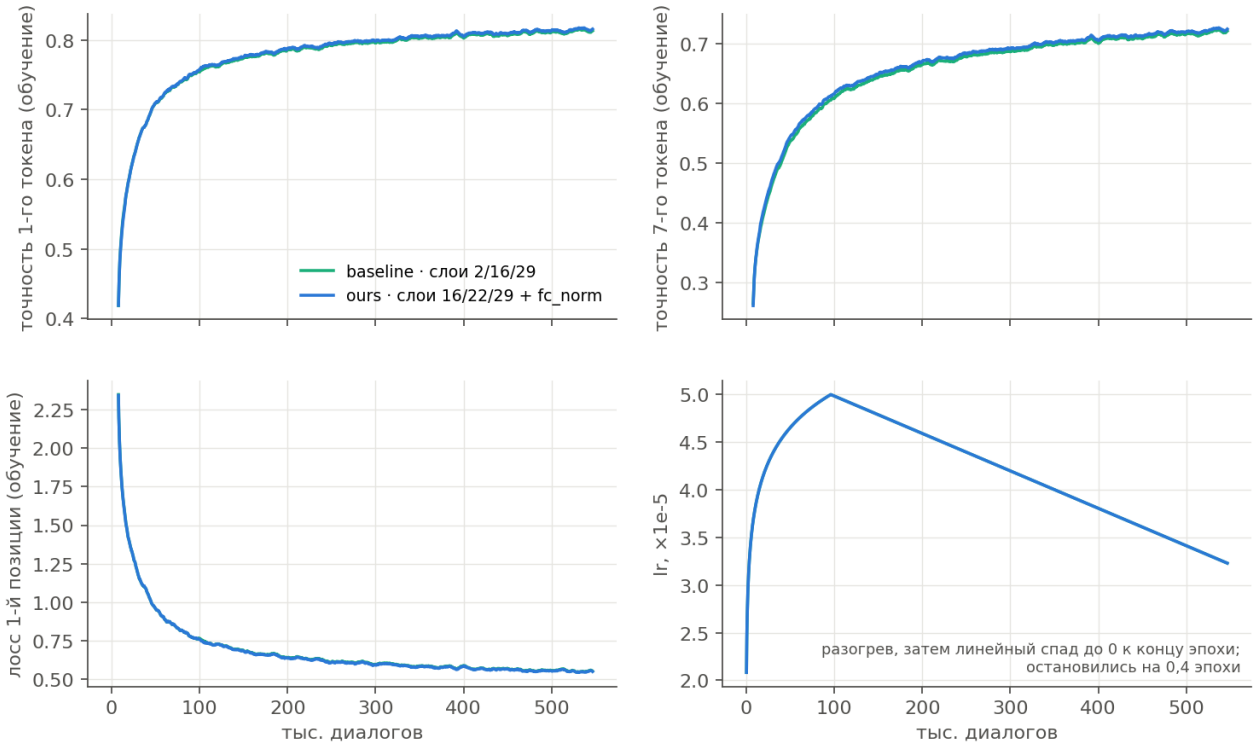


Рис. 4: Кривые обучения (скользящее среднее по 20 строкам лога) и расписание lr. Кривые обоих вариантов почти совпадают; разницу видно в парном сравнении (рис. 11).

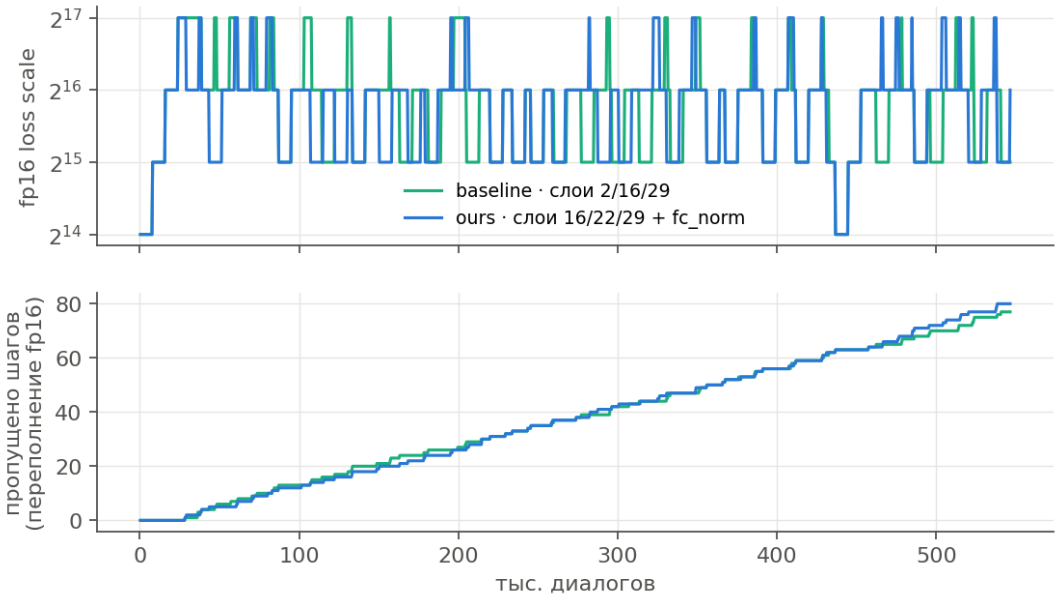


Рис. 5: Устойчивость fp16: масштаб loss scaling и число пропущенных шагов из-за переполнения. 77 и 80 пропусков из 68 325 шагов (0,1 %), ни одного нечислового значения.

2.1 Эпизод с NeMo

Первоначально ours должен был учиться рецептом NeMo AutoModel. Пять попыток, четыре срыва при lr от 2e-4 до 2e-5; ни одна не прошла дальше шага ≈930. Причина — рост логитов внимания q·k: внутри срыва градиенты по `q_proj`, `k_proj` и `fc` в 14–27 раз больше, чем у здоровой модели. Главное отличие от кода авторов — инициализация: веса драфта NeMo стартуют втрое крупнее. Подробный разбор — в отдельном отчёте «Почему ours больше не учится на NeMo». Решение: оба варианта учатся кодом авторов, сравнение стало чище.

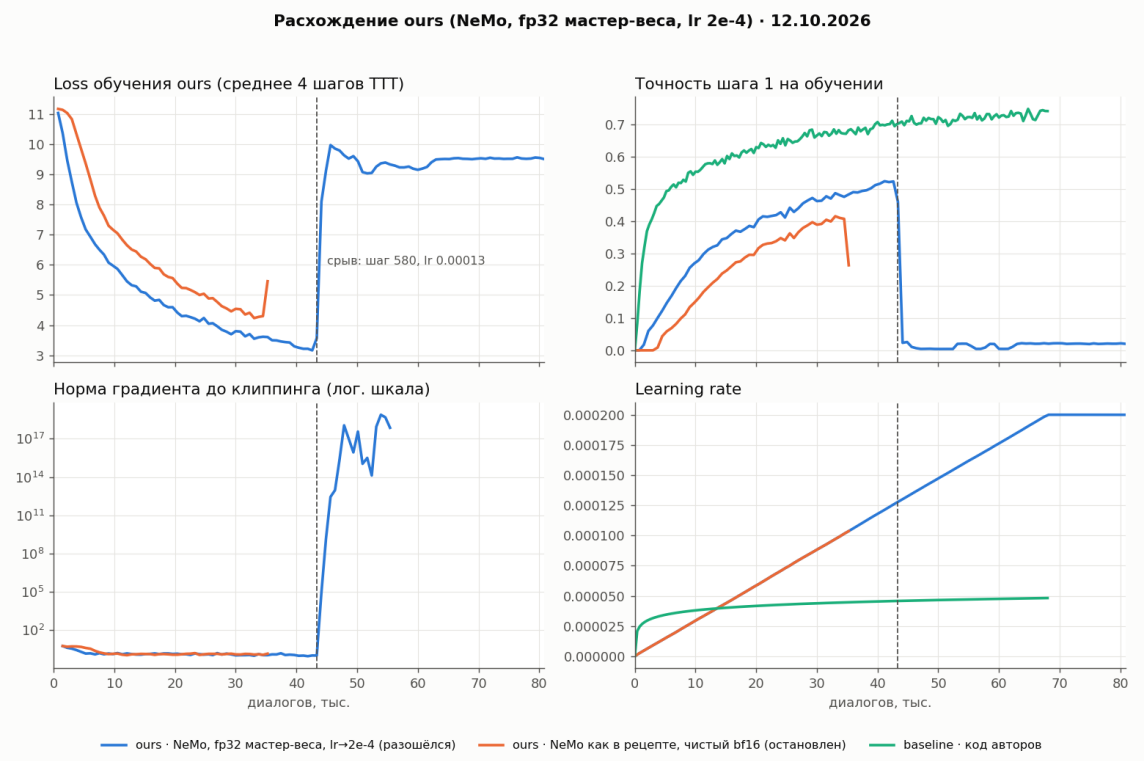


Рис. 6: Срывы ours на рецепте NeMo, 12.10; для сравнения — baseline на коде авторов.

3 Валидация: ours впереди с самого начала

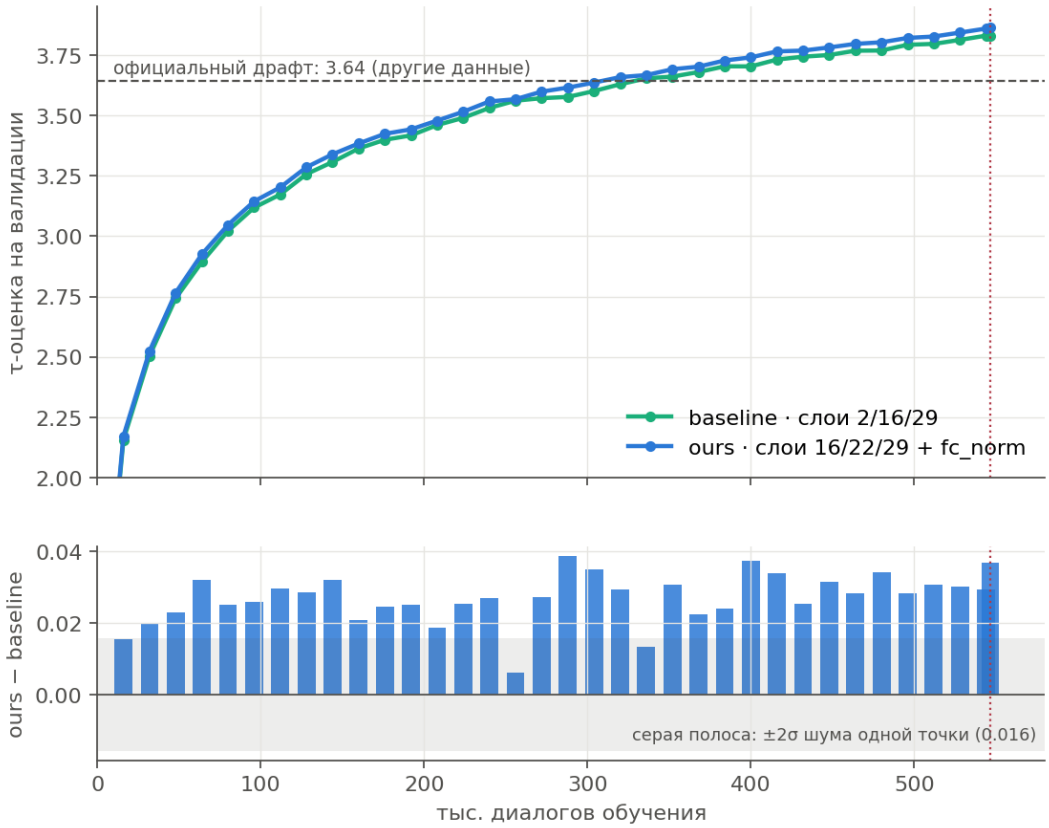


Рис. 7: τ-оценка на валидации. Снизу — разница ours − baseline; серая полоса — ±2σ шума одной точки (σ = 0,0078).

Таблица 2: τ-оценка на валидации при равном числе диалогов (выборка точек).

тыс. диалогов	ours	baseline	ours − baseline
16	2,171	2,156	+0,016
64	2,925	2,893	+0,032
128	3,285	3,256	+0,029
256	3,567	3,560	+0,006
384	3,727	3,703	+0,024
512	3,826	3,796	+0,030
546,6	**3,865**	3,828	**+0,037**

Знак разницы не меняется ни разу: 35 точек из 35. Средняя разница по участкам: +0,025 (до 128 тыс.), +0,026 (128–320 тыс.), +0,029 (320–546,6 тыс.).

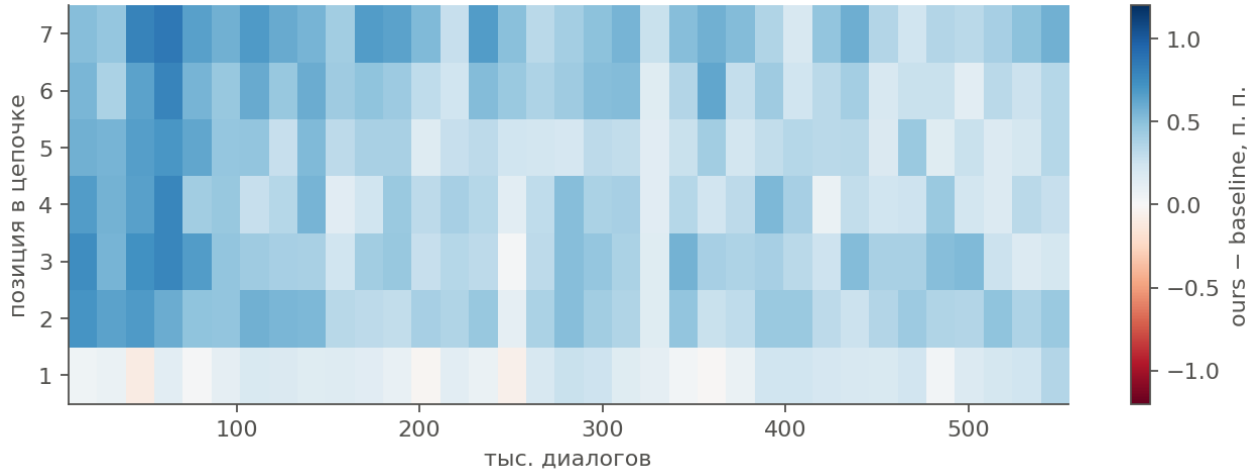


Рис. 8: Разница точности ours – baseline на валидации по позициям цепочки и по ходу обучения. Синий — ours точнее. Положительных клеток 98 %.

Где выигрыш. На первой позиции, то есть для первого угаданного токена, варианты почти равны. Выигрыш ours — на позициях 2–7, когда драфт угадывает дальше по собственным предсказаниям. Это согласуется с тем, что более глубокие слои цели несут больше информации о дальнейшем тексте.

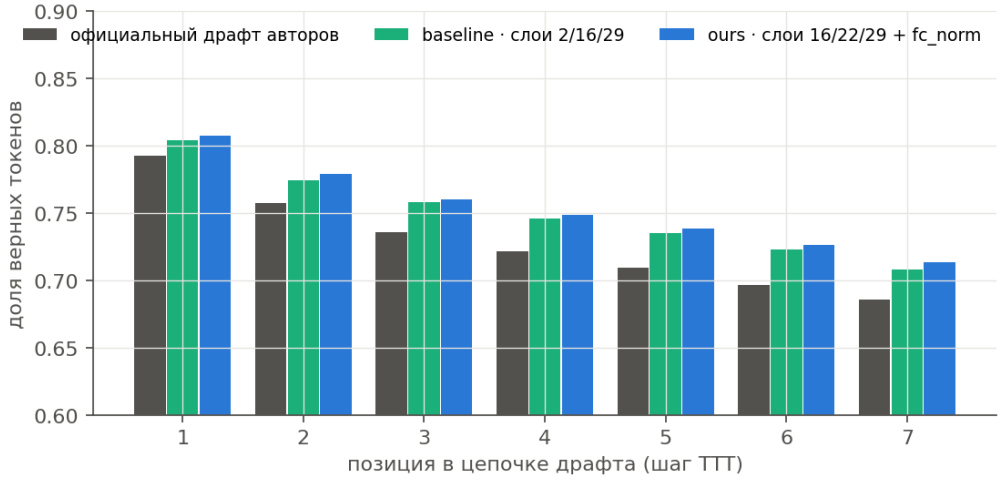


Рис. 9: Точность по позициям в точке сравнения; официальный драфт — на тех же 400 диалогах.

3.1 Экономия данных



Рис. 10: Сколько процентов диалогов baseline нужно сверх ours, чтобы выйти на ту же τ-оценку (по гладкой кривой baseline).

Разрыв растёт: 3–10 % в первой половине, 8–15 % в последних восьми точках. Свои 3,865 ours набрал за 546,6 тыс. диалогов; baseline до них не дошёл.

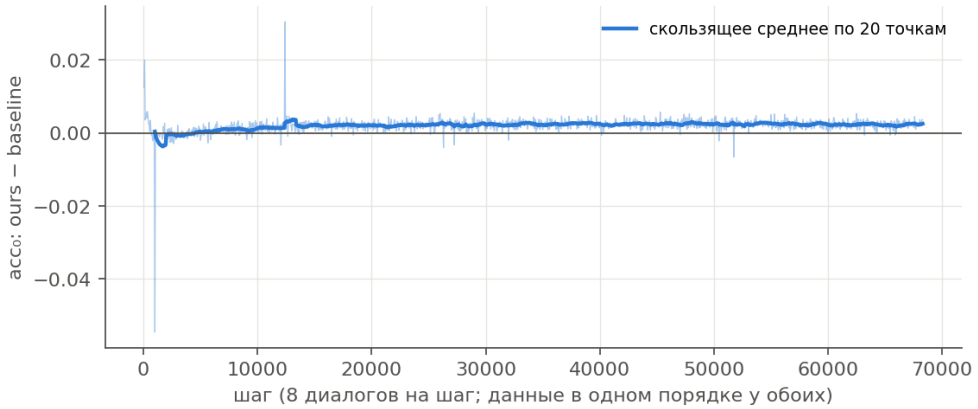


Рис. 11: Парное сравнение на обучении: оба варианта видят одни и те же диалоги на одних шагах. Точность первого токена у ours выше на 0,24 п.п. в среднем, в 99 % строк лога.

4 Реальная τ и ускорение

Таблица 3: Реальная τ: SGLang 0.5.9, дерево 8/10/60, T = 0, 80 вопросов. В скобках — код авторов. Последние строки — разница ours − baseline в SGLang и её 95 % интервал.

драфт	MT-bench	HumanEval	GSM8K	Alpaca	CNN/DM	среднее
официальный	**6,11** (6,18)	6,69 (6,74)	6,18 (6,26)	**6,63**	**5,34**	**6,19**
baseline	4,95 (4,98)	7,54 (7,56)	**7,41** (7,43)	4,71	3,96	5,71
ours	5,03 (5,04)	**7,56** (7,61)	7,39 (7,41)	4,80	3,97	5,75
ours − baseline	**+0,080**	+0,024	−0,015	**+0,087**	+0,011	**+0,037**
95 % интервал	[+0,05; +0,11]	[−0,02; +0,07]	[−0,09; +0,06]	[+0,04; +0,14]	[−0,03; +0,05]	[+0,014; +0,060]

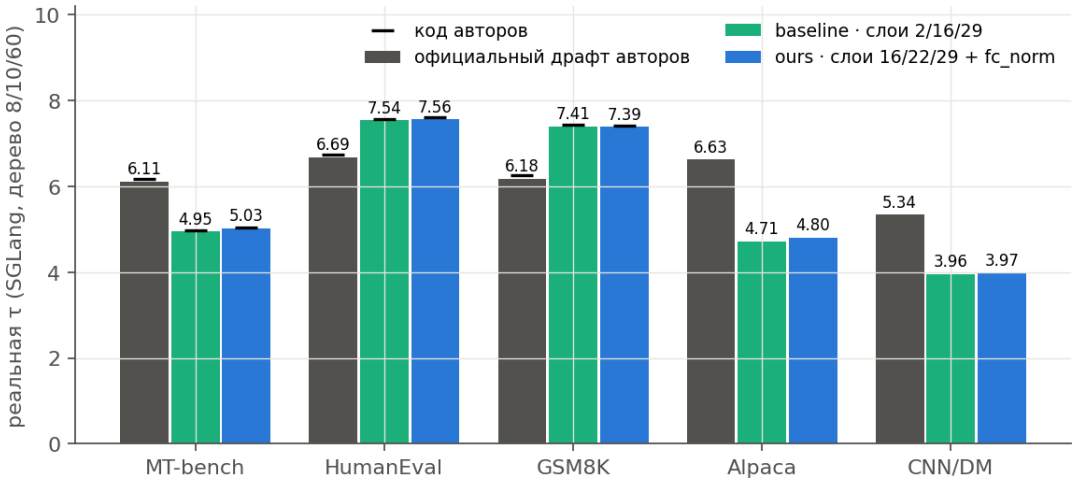


Рис. 12: Реальная τ по бенчмаркам. Столбцы — SGLang, чёрные штрихи — код авторов.

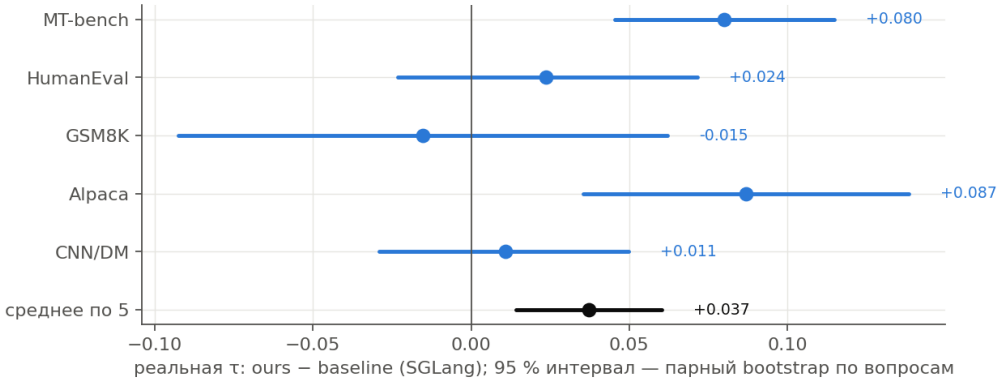


Рис. 13: Реальная τ: ours – baseline, точка и 95 % интервал.

- **Ours лучше в среднем, и это значимо:** +0,037 τ, или +0,65 %. Валидация предсказала столько же в абсолютных единицах: +0,037 τ-оценки.
- **Выигрыш — в диалогах:** MT-bench +0,08, Alpaca +0,09, оба значимы. На коде и математике τ уже высокая (7,4–7,6, почти потолок дерева 8), на пересказе драфт упирается в данные; там разница в пределах шума.
- **Стенды согласованы:** SGLang и код авторов совпадают до 0,08 для всех трёх драфтов (для ours — после исправления бага, раздел 7).
- **Официальный драфт — другие данные.** Его учили на ShareGPT и UltraChat и намного дольше; наши — на PerfectBlend, где много кода и математики. Поэтому наши драфты за 0,4 эпохи обгоняют официальный на HumanEval и GSM8K (+0,9 и +1,2), но отстают на диалогах и пересказе (−1,1…−1,9).

Таблица 4: Ускорение в SGLang к обычной генерации на той же свободной H200 (batch 1). В скобках — токенов в секунду.

драфт	MT-bench	HumanEval	GSM8K	Alpaca	CNN/DM	среднее
официальный	**2,95×** (532)	3,36× (615)	3,15× (583)	**3,38×** (621)	**2,46×** (426)	**3,06×**
baseline	2,57× (463)	3,76× (687)	**3,77×** (697)	2,57× (473)	1,85× (321)	2,90×
ours	2,59× (466)	3,76× (687)	3,75× (694)	2,61× (479)	1,86× (322)	2,91×
обычная генерация	180	183	185	184	174	

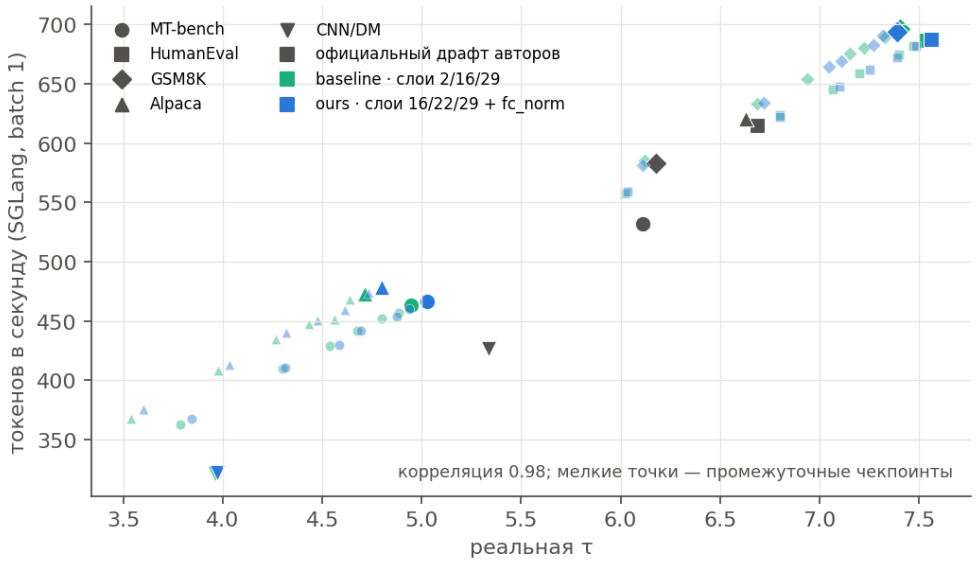


Рис. 14: Скорость повторяет τ: все драфты и бенчмарки, включая промежуточные чекпоинты, ложатся на одну зависимость. `fc_norm` на скорость не влияет.

4.1 Как устроен выигрыш

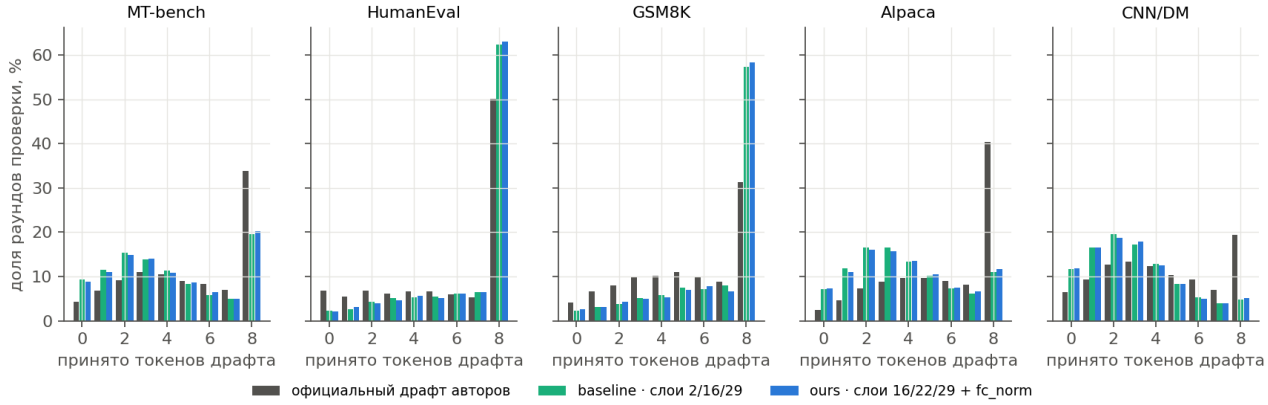


Рис. 15: Сколько токенов драфта принимается за раунд проверки (0–8). У официального драфта на диалогах намного чаще принимается вся ветка (8); у наших — на коде и математике.

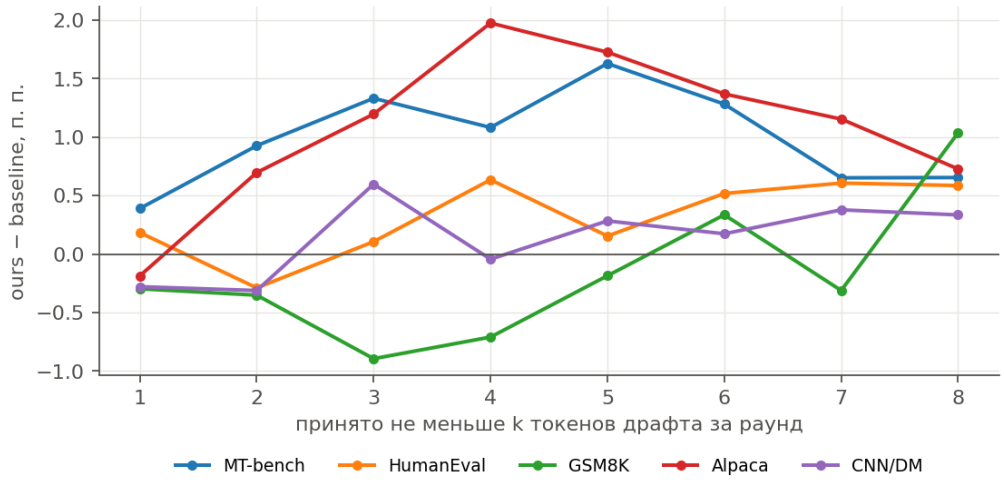


Рис. 16: Доля раундов, где принято не меньше k токенов: ours − baseline. На MT-bench и Alpaca ours переносит раунды из коротких принятий (1–2 токена) в средние и длинные (3–7) — на 1–2 п.п.

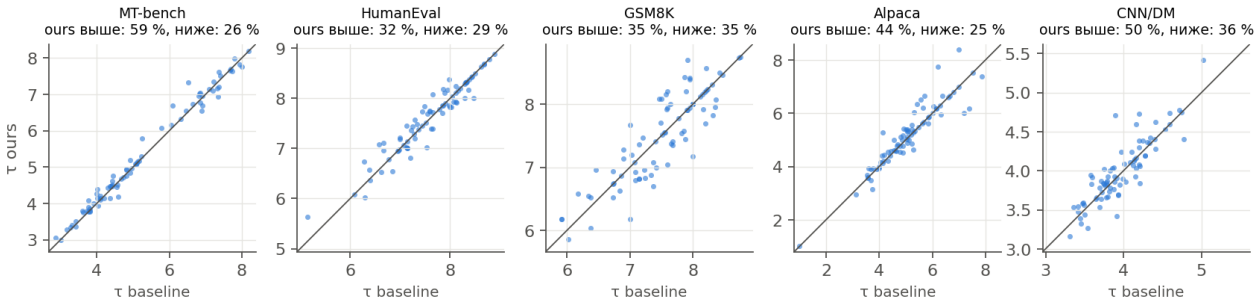


Рис. 17: τ по отдельным вопросам: ours против baseline. На MT-bench ours выше на 59 % вопросов и ниже на 26 %; на HumanEval и GSM8K много равенств — оба драфта часто принимают всю ветку.

5 Реальная τ по ходу обучения

Каждые 10 000 шагов (80 тыс. диалогов) оба варианта сохраняли веса. Мы прогнали каждый такой чекпоинт в SGLang (с исправлением, раздел 7) на MT-bench, HumanEval, GSM8K и Alpaca — по 80 вопросов, как в итоговой оценке.

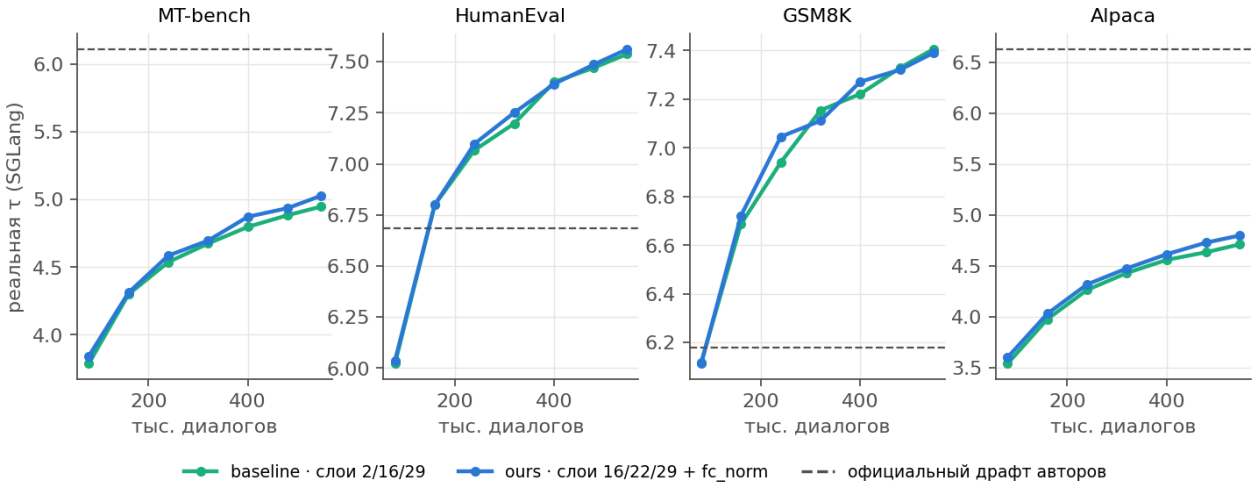


Рис. 18: Реальная τ на промежуточных чекпоинтах; пунктир — официальный драфт. На коде и математике наши драфты обгоняют его уже к 160 тыс. диалогов.

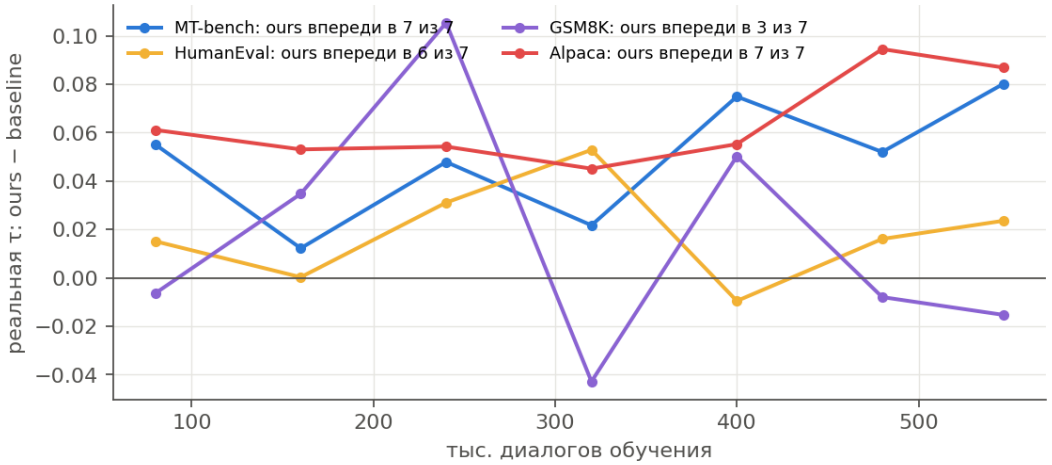


Рис. 19: Реальная τ: ours − baseline на каждом чекпоинте.

- **На диалогах ours впереди на каждом чекпоинте:** MT-bench и Alpaca — 7 из 7, разница +0,01…+0,09. Вероятность такого ряда при отсутствии эффекта — $2^{-7} < 0{,}01$ для каждого бенчмарка. На HumanEval ours впереди в 6 из 7, на GSM8K — в 3 из 7: там разница в пределах шума.
- **Код и математика быстро упираются в потолок дерева:** τ 7,4–7,6 при максимуме 9. Официальный драфт наши обгоняют на HumanEval и GSM8K уже к 160 тыс. диалогов.
- **Диалоги растут медленно и далеки от официального драфта** (4,8–5,0 против 6,1–6,6) — снова довод в пользу диалоговых данных.

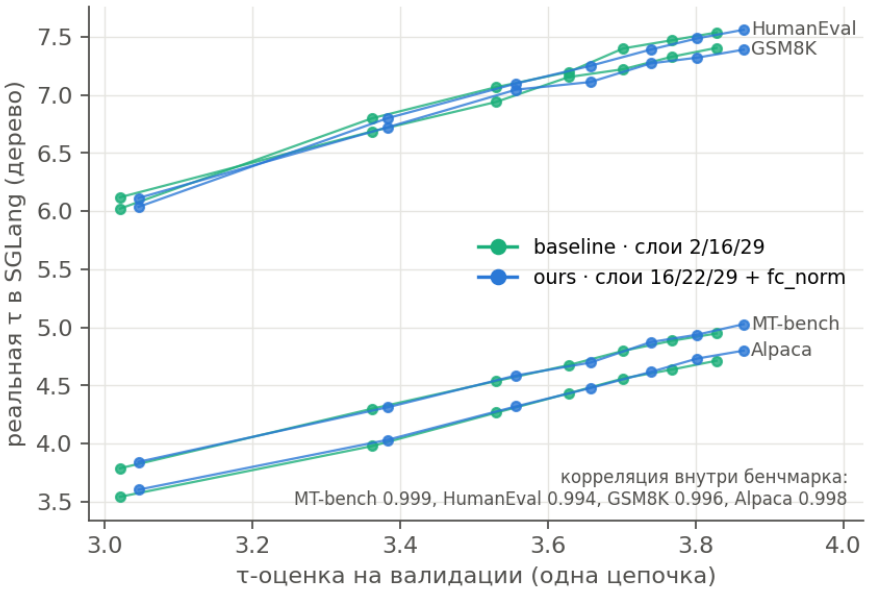


Рис. 20: τ-оценка на валидации против реальной τ на тех же чекпоинтах. Внутри каждого бенчмарка зависимость почти линейная (корреляция 0,994–0,999).

Дешёвая метрика работает. τ-оценка на валидации (одна цепочка, 400 диалогов, несколько минут на GPU) упорядочивает чекпоинты так же, как реальная генерация: корреляция внутри бенчмарка 0,994–0,999. Значит, решения по ходу обучения можно принимать по ней, а дорогую реальную оценку запускать только в конце. Сравнивать по ней драфты, обученные на *разных* данных, нельзя: официальный драфт на нашей валидации ниже наших, а на реальной генерации выше.

6 Почему наши слои помогают: что драфт берёт из каждого слоя

Вход драфта — $\mathrm{fc}([x_a; x_b; x_c]) = W_a x_a + W_b x_b + W_c x_c$, где x_k — состояние целевой модели на входе слоя k. Мы прогнали 64 диалога валидации (25 тыс. токенов ответов) через целевую модель и для каждого сохранённого чекпоинта (каждые 10 000 шагов) посчитали, какую долю энергии выхода fc даёт каждое слагаемое $W_k x_k$ (у ours — после `fc_norm`).

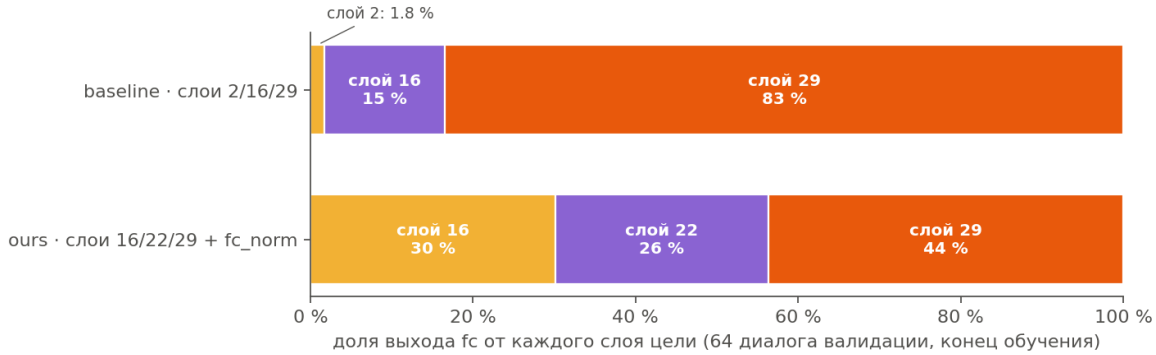


Рис. 21: Доля выхода fc от каждого слоя цели в конце обучения.

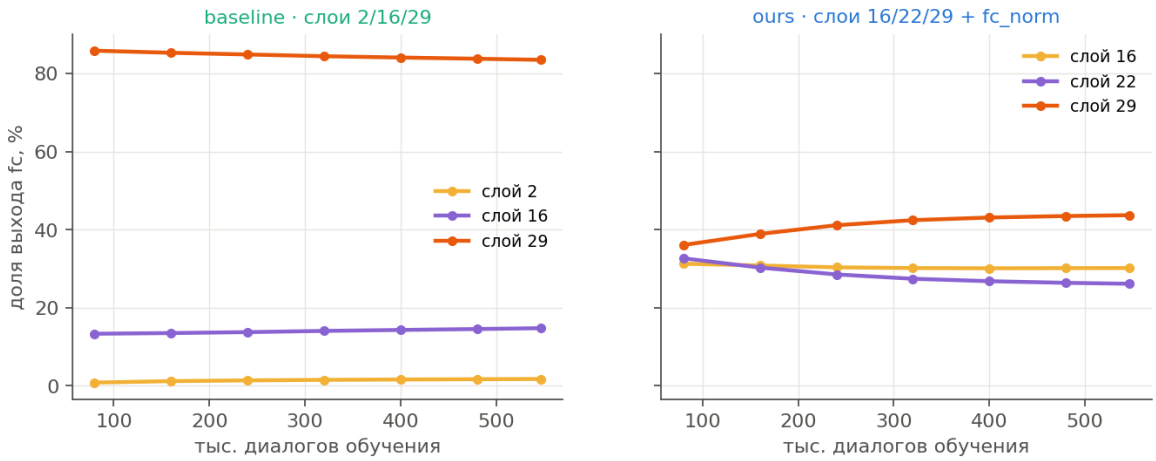


Рис. 22: Те же доли по ходу обучения.

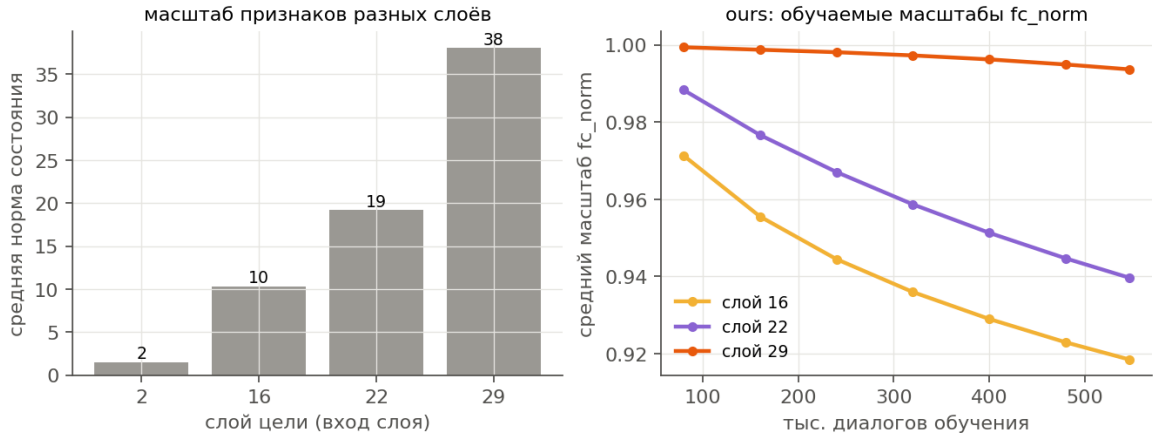


Рис. 23: Слева — средняя норма состояний целевой модели на входе слоёв 2, 16, 22, 29. Справа — обучаемые масштабы `fc_norm` у ours.

- **Baseline почти не пользуется слоем 2:** 1,8 % выхода fc. Слой 16 даёт 15 %, слой 29 — 83 %. Это прямо подтверждает вывод исследования выбора слоёв, теперь на драфте, обученном с нуля.
- **Причина — масштаб.** Норма состояний растёт с глубиной: 1,5 на входе слоя 2, 10 у слоя 16, 19 у 22 и 38 у 29. Без нормировки fc должен компенсировать разницу в 25 раз. Baseline пытается: блок весов для слоя 2 у него самый крупный (норма 69 против 45 у слоя 29), но этого не хватает.
- **У ours все три слоя работают:** 30 / 26 / 44 %. `fc_norm` приводит признаки к одному масштабу, и fc распределяет вес по содержанию, а не по норме. По ходу обучения доля слоя 29 растёт с 36 до 44 %, а масштабы `fc_norm` для слоёв 16 и 22 плавно снижаются (до 0,92 и 0,94).
- **Отсюда и выигрыш на дальних позициях** (рис. 8): драфт ours реально получает три разных источника информации, а baseline — по сути полтора.

7 Баг SGLang 0.5.9: слои цели в CUDA-графах

Первый замер ours в SGLang (14.10) дал τ на 13 % ниже, чем код авторов на том же драфте: 4,36 против 5,04 на MT-bench. Для официального драфта и baseline стенды совпадали. Причина — порядок инициализации в SGLang 0.5.9:

1. Слои цели SGLang читает из конфига драфта (`eagle_aux_hidden_state_layer_ids`).
2. Но `CudaGraphRunner` перед записью графов вызывает `set_eagle3_layers_to_capture()` без аргументов, и в графы decode и проверки дерева записываются слои по умолчанию 2, $N/2$, $N-3$.
3. Слои из конфига выставляются после этого и действуют только в prefill, который идёт без графов.

Драфты со слоями по умолчанию ошибки не замечают. Ours в каждом раунде проверки получал признаки слоёв baseline.

Исправление — в нашем внешнем пакете `sglang_eagle31`, который SGLang и так загружает ради `fc_norm`: вызов без аргументов берёт слои из конфига драфта, которые модель уже прочитала до записи графов. Проверка — строка `target layers to capture (output ids) [15, 21, 28], also in CUDA graphs` в логе сервера.

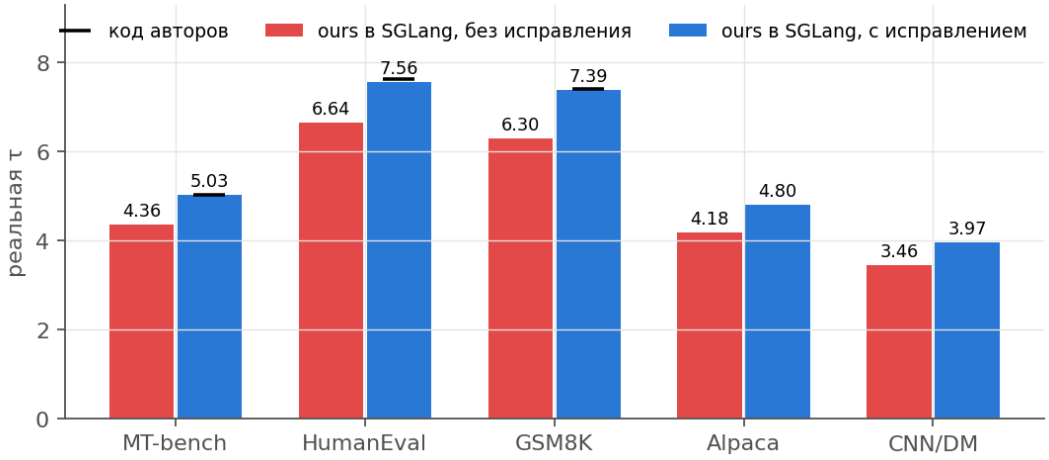


Рис. 24: Реальная τ ours в SGLang до и после исправления; чёрные штрихи — код авторов.

Вывод для всех, кто переносит драфты со своими слоями в SGLang: сверять τ с кодом авторов на каждом новом драфте. Баг молчаливый: сервер работает, ускорение есть, но заметно меньше, чем должно быть.

8 Прогноз: сколько ещё учить

Кривые τ-оценки хорошо описываются законом $\hat{\tau}(n) = a - b\,n^{-c}$, где n — тысячи диалогов, $c \approx 0{,}22$. Мы подогнали его по точкам начиная с 32, 64 и 128 тыс.; разброс подгонок — грубая мера неопределённости.

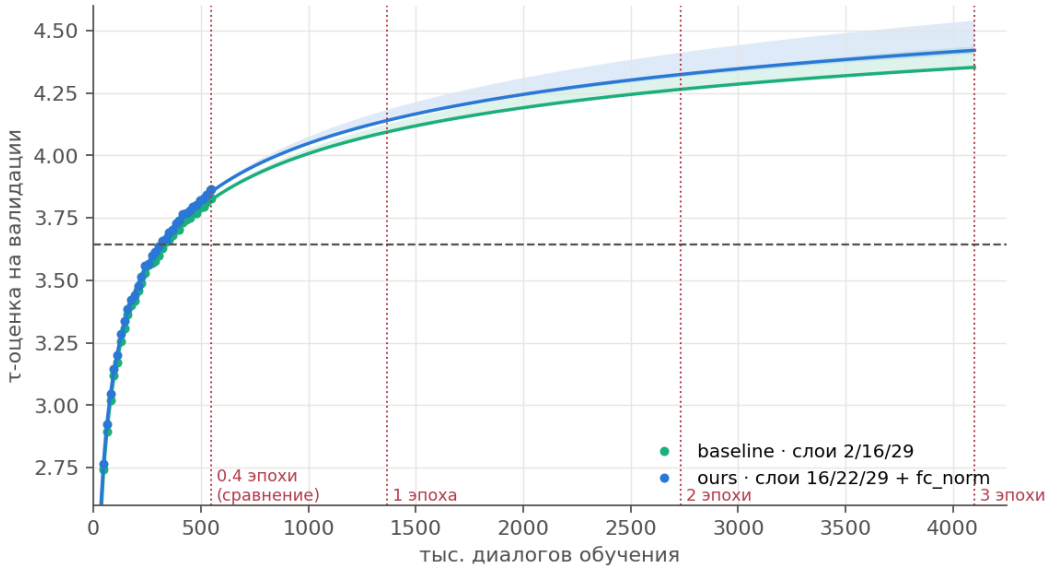


Рис. 25: Экстраполяция τ-оценки на 1–3 эпохи; пунктир — официальный драфт на нашей валидации.

Таблица 5: Прогноз τ-оценки и число диалогов до заданного уровня.

	0,4 эпохи (546,6 тыс.)	1 эпоха (1,37 млн)	2 эпохи (2,73 млн)	3 эпохи (4,10 млн)
baseline	3,83 (замер)	4,09 (4,09–4,12)	4,26 (4,26–4,33)	4,35 (4,35–4,44)
ours	3,87 (замер)	**4,14** (4,13–4,18)	**4,32** (4,31–4,41)	**4,42** (4,41–4,54)
	до $\hat{\tau} = 3{,}9$	до 4,0	до 4,1	до 4,2
baseline, млн диалогов	0,68–0,70	0,92–0,98	1,27–1,40	1,76–2,08
ours, млн диалогов	0,62–0,63	**0,81–0,87**	**1,08–1,21**	**1,44–1,74**

- **Отдача от данных падает медленно:** каждое удвоение добавляет 0,22 → 0,19 → 0,17 у ours. Драфт далёк от насыщения, разрыв с baseline по прогнозу растёт (+0,05 к одной эпохе, +0,07 к трём).
- **Прогноз скорее занижен:** в точке сравнения lr ещё 3,2e-5, а отжиг lr к концу эпохи обычно добавляет сверх кривой.
- **Цена:** до конца эпохи около 23,5 ч H200 на вариант (три 8-часовые сессии); три эпохи с нуля — около 118 ч.
- **На диалогах главный рычаг — данные.** Чтобы обогнать официальный драфт на MT-bench и Alpaca, нужно добавить ShareGPT и UltraChat с ответами цели.

9 Выводы и что дальше

1. **Слои 16/22/29 с `fc_norm` лучше слоёв статьи и при обучении с нуля:** +0,037 реальной τ в среднем, +0,08–0,09 на диалогах, при той же скорости. Выигрыш виден с первых тысяч диало-

гов и растёт.

2. **Механизм понятен:** без нормировки драфт почти игнорирует нижний слой, потому что его признаки в 25 раз меньше по норме; `fc_norm` выравнивает масштабы, и все три слоя начинают работать.
3. **Перенос в движки:** экспорт для SGLang и NeMo/vLLM готов (`export.py`); в SGLang 0.5.9 нужен наш пакет `sglang_eagle31` — он добавляет `fc_norm` и чинит захват слоёв.
4. **Дальше:** дотянуть до полной эпохи (отжиг lr); добавить диалоговые данные; отправить исправление захвата слоёв в SGLang upstream; проверить загрузку экспорта в NeMo/vLLM замером τ.

A Хронология финальной стадии

12–15 октября

12.10 15:26 — старт обоих вариантов кодом авторов.
12.10 16:00 – 13.10 00:12 — ours на рецепте NeMo: пять попыток, срывы; решение вернуть ours на код авторов.
13.10 – 14.10 — 6 сессий SLURM; мониторинг через Telegram-бота и ClearML.
14.10 13:22 — baseline дошёл до шага 68 325. Итоговая оценка официального драфта и baseline.
14.10 17:03 — узел перестал отвечать по сети, работа на нём продолжилась.
14.10 17:42 – 18:22 — ours дошёл до шага 68 325; итоговая оценка ours, обычной генерации и чистые скорости.
15.10 12:20 — доступ восстановлен; найдено расхождение SGLang и кода авторов для ours и его причина — баг захвата слоёв.
15.10 12:23 – 14:01 — ours перемерен с исправлением; вклад слоёв и реальная τ на 12 промежуточных чекпоинтах.

B Данные и воспроизводимость

Всё лежит в репозитории проекта, в папке `big-training/`:

- `results/llama31-8b/` — данные:
  - `<вариант>/val.jsonl`, `metrics.jsonl`, `events.jsonl`, `status.json` — полные логи обучения;
  - `final_eval/` — реальная τ по вопросам с гистограммами принятия для всех драфтов, промежуточных чекпоинтов, обычной генерации и замера без исправления SGLang, итоги кода авторов;
  - `milestones/contributions.json` — вклад слоёв;
  - `data_stats.json`, `reference.json`, `results.json`, `extra.json`, `tables.md`.
- `code/collect_results.py` собирает эти данные из копии логов сервера.
- `code/make_results_figures.py` и `code/make_extra_figures.py` строят все графики отчёта в `figures/`.
- `code/milestone_analysis.py` — вклад слоёв и реальная τ на чекпоинтах (на GPU).

Полная копия логов сервера без весов (около 50 МБ, с текстами ответов) хранится локально у команды.